Introduction to Deep Learning

Session 9: Convolutional Neural Networks

May 2026

Magda Gregorová - magda.gregorova@thws.de

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

Lecture Overview

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

1. Why Not MLPs for Images?

2. Convolution Operation

Images in Computers

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

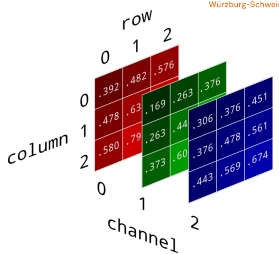


Image: 3D tensor of shape $C \times H \times W$

- Each entry: pixel intensity, integer 0–255 or float 0–1,
- 2^8 possible values $\sim$ 8 bits
- **Grayscale** ($C = 1$): single channel, 0 = black, 255 = white
- **RGB** ($C = 3$): red, green, blue — standard cameras, screens

Other formats:

- RGBA ($C = 4$): RGB + α transparency
- CMYK ($C = 4$): print format
- Medical/hyperspectral: many channels, often 16-bit depth

DL mostly **RGB and grayscale** — other formats typically converted before use

Images as Data

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Image sizes — from benchmarks to real world:

Source	Size	Channels	Pixels
MNIST	28×28	1	784
ImageNet	224×224	3	150 528
Phone camera	4000×3000	3	36 000 000
Full HD screen	1920×1080	3	6 220 800
Medical (CT slice)	512×512	1	262 144

MLP approach: flatten image to vector $\mathbf{x} \in \mathbb{R}^{H \cdot W \cdot C}$, apply fully connected layers

ImageNet: first hidden from $\mathbb{R}^{150\,528} \to \mathbb{R}^{1024} \Rightarrow \approx 154$M parameters — one layer only!

How to handle high-dimensional data efficiently?

How to Look at Images

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Human vision:

- Looks for local patterns — edges, corners, textures
- Same pattern recognised anywhere in the visual field
- Simple features combine into complex ones

Technical equivalent:

- **Local connectivity** — connect to small patches only
- **Parameter sharing** — same filter at every location
- **Hierarchical features** — edges → shapes → objects

Three key properties: **locality**, **translation invariance**, **parameter efficiency**

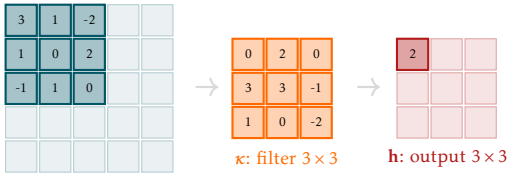
These define the **convolutional layer**

Convolution Operation

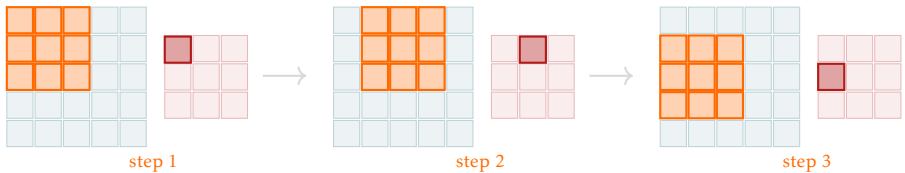
Convolution — The Operation

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Single step: dot product of local patch and filter → one output value



Full operation: same filter slides across input — each position produces one output value

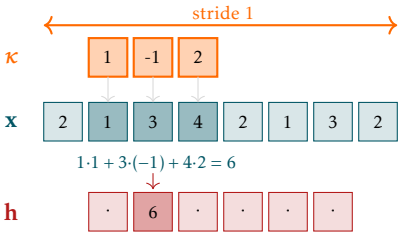


All 3 requirements: local connectivity, parameter sharing, translation invariance

1D Convolution

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Filter $\kappa \in \mathbb{R}^k$ slides over signal $\mathbf{x} \in \mathbb{R}^n$ — at each position i, dot product with local patch



$$(\mathbf{x} * \boldsymbol{\kappa})[i] = \sum_{j=0}^{k-1} \mathbf{x}[i+j]\,\boldsymbol{\kappa}[j]$$

- i — output position; j — index within filter
- Dot product of filter with local patch

Example: at position $i = 1$:

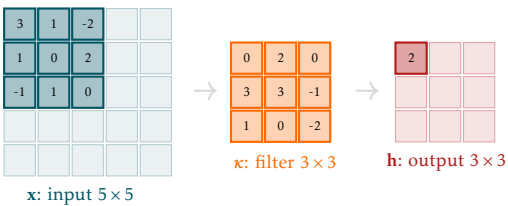
$$1\cdot 1 + 3\cdot(-1) + 4\cdot 2 = 6$$

2D Convolutiona

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

Extend to 2D: filter $\kappa \in \mathbb{R}^{k \times k}$ slides over image $\mathbf{X} \in \mathbb{R}^{H \times W}$:

$$(\mathbf{X} * \kappa)[i, j] = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[i+p, j+q]\, \kappa[p, q]$$

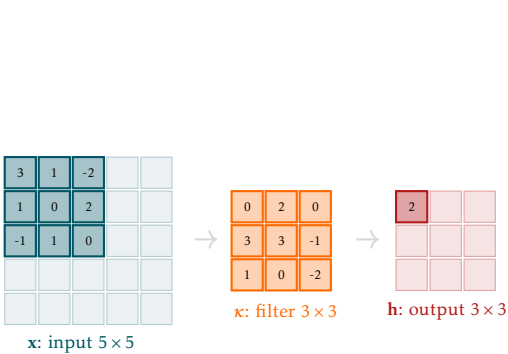


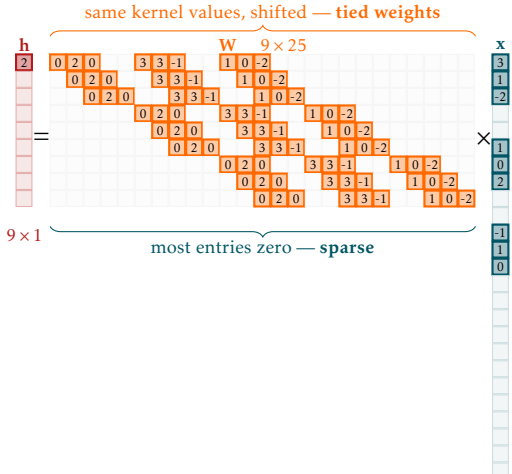
- (i, j) — output position
- (p, q) — index within filter
- Two sums: over rows and columns of filter

Output size: $(H - k + 1) \times (W - k + 1)$

Convolution as Matrix Multiplication

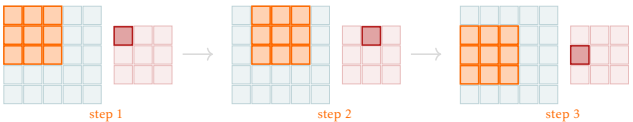
Flatten input and output — convolution becomes matrix multiplication

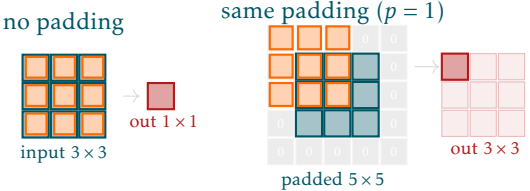




Padding

Problem: convolution shrinks spatial dimensions — each layer loses $k-1$ pixels





Valid padding ($p = 0$)

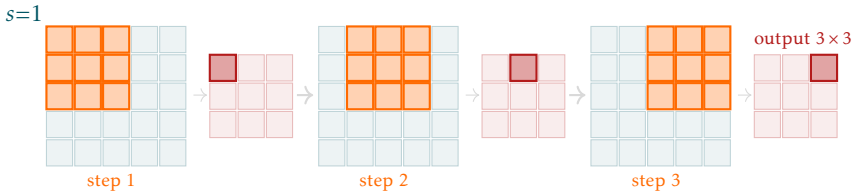
- No zeros added
- Output shrinks: $(H-k+1) \times (W-k+1)$
- Spatial information lost at borders

Same padding ($p = (k-1)/2$)

- Zeros added around border
- Output same size as input
- Standard choice in most architectures

Stride

Stride s: filter κ moves s pixels at each step instead of 1



Stride 1: κ moves one step a time

Stride 2: κ skips one - output halved

General output size: $\left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$

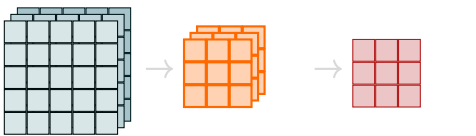
Multiple Input Channels

RGB input: C_{in} channels — filter must have same depth C_{in}

Operation: filter has C_{in} slices — each slice convolves with one input channel

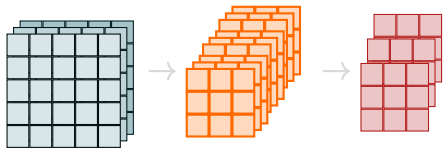
$$(\mathbf{X} * \boldsymbol{\kappa})[i,j] = \sum_{c=0}^{C_{in}-1} \underbrace{\sum_{p=0}^{k-1}\sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q]\, \boldsymbol{\kappa}[c,p,q]}_{\text{conv with channel } c}$$

- Inner double sum: convolution of filter slice c with input channel c
- Outer sum over c: results summed across all channels
- One filter → one output feature map



Multiple Filters

C_{out} **filters:** each produces one feature map — output has C_{out} channels



- Each filter detects a different pattern
- Output tensor: $C_{out} \times H' \times W'$
- Parameters per layer: $C_{out} \times C_{in} \times k \times k + C_{out}$ (with bias)

In PyTorch: `nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)`

Receptive Field

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Receptive field: region of the input that influences a given neuron's output

- Layer 1 (3×3 filter): receptive field = 3×3
- Layer 2 (3×3 filter): receptive field = 5×5
- Layer L (3×3 filters): receptive field = $(2L+1) \times (2L+1)$

Key insight: deep networks have large receptive fields despite small local filters

- Early layers detect local features (edges, corners)
- Later layers combine local features into complex patterns (eyes, wheels, faces)
- Hierarchical feature learning — the hallmark of deep networks

Two 3×3 filters have the same receptive field as one 5×5 filter but fewer parameters: $2 \times 9 = 18$ vs 25